

Daraja (M-Pesa) Production Go-Live Runbook

This runbook takes Fynl-DCP's Safaricom Daraja integration from **sandbox** to **production**. Going live is a **config change, not a code change**: you flip a single environment variable (`DARAJA_ENVIRONMENT`) and supply real production credentials, then restart the service.

Golden rule: the default is `DARAJA_ENVIRONMENT=sandbox` . Production is only ever reached by a deliberate, manual change. Do **not** flip to production until every step in the pre-go-live checklist below is done.

1. How the environment switch works

A single setting drives everything:

DARAJA_ENVIRONMENT	Daraja base URL used everywhere
<code>sandbox</code> (default)	<code>https://sandbox.safaricom.co.ke</code>
<code>production</code>	<code>https://api.safaricom.co.ke</code>

The base URL is derived from this single setting in `backend/app/core/config.py` (`Settings.DARAJA_BASE_URL`) and `backend/app/services/mpesa.py` (`DarajaCreds.base_url`). It is applied to **every** Daraja call — OAuth token, B2C payment request, STK push, and C2B URL registration. There is **no hardcoded sandbox host in any request path**.

Every Daraja credential is read from the environment (`backend/.env` / `process.env`), never hardcoded:

Setting	Purpose
<code>DARAJA_ENVIRONMENT</code>	<code>sandbox</code> <code>production</code> — the go-live switch
<code>DARAJA_CONSUMER_KEY</code>	OAuth app consumer key
<code>DARAJA_CONSUMER_SECRET</code>	OAuth app consumer secret
<code>DARAJA_SHORTCODE</code>	B2C paybill / shortcode (PartyA)
<code>DARAJA_INITIATOR_NAME</code>	B2C initiator (API operator) username
<code>DARAJA_SECURITY_CREDENTIAL</code>	Initiator password encrypted with Safaricom's cert
<code>DARAJA_PASSKEY</code>	Lipa-na-M-Pesa passkey (STK push only)
<code>DARAJA_CALLBACK_BASE_URL</code>	Public host Safaricom posts webhooks back to
<code>MPESA_CALLBACK_TOKEN</code>	Unguessable path segment on every callback URL

Fail-closed safety guard

When `DARAJA_ENVIRONMENT=production`, a B2C payout requires the full set of live credentials: `DARAJA_CONSUMER_KEY`, `DARAJA_CONSUMER_SECRET`, `DARAJA_SHORTCODE`, `DARAJA_INITIATOR_NAME`, `DARAJA_SECURITY_CREDENTIAL`. If **any** of these is missing/empty, the app:

1. logs a clear error (`finyl.mpesa` — lists which fields are missing, **no secret values**), and
2. **refuses the payout** (`DarajaNotConfigured` → HTTP 422 to the caller).

It will **never** silently simulate a payout or fall back to the sandbox host on a live request. (In sandbox the behaviour is unchanged: a real sandbox call when credentials are present, a simulated async acknowledgement when they are not.)

On boot the service logs one secret-free line (logger `finyl.startup`), e.g.:

```
Daraja environment: sandbox (base_url=https://sandbox.safaricom.co.ke, status=...);
consumer key/secret configured: yes; shortcode configured: yes; initiator configured:
yes;
security credential configured: yes; passkey configured: yes
```

Use it to confirm which environment the service actually came up in.

1b. Prove the sandbox link first (recommended)

Before going anywhere near production, confirm Fynl-DCP can actually talk to Safaricom in the **sandbox**. This needs only the two sandbox OAuth values.

1. On <https://developer.safaricom.co.ke>, open (or create) an app and copy its **sandbox** Consumer Key and Consumer Secret (the “Lipa Na M-Pesa Sandbox” product). The sandbox shortcode (174379) and test passkey are public and filled in automatically by the diagnostic below.
2. Run the self-contained diagnostic harness — it exercises the exact `app.services.mpesa` code the API uses, without touching the DB:

```
bash
cd backend
DARAJA_CONSUMER_KEY=xxxx DARAJA_CONSUMER_SECRET=yyyy \
python3 scripts/test_daraja_sandbox.py
```

It runs five checks and prints a clear PASS/FAIL for each:

#	Check	Proves
0	Credentials present	Env is wired
1	Reachability	Network path to <code>sandbox.safaricom.co.ke</code>
2	OAuth token	Consumer key/secret are valid
3	Token cache reuse	No redundant token calls per request
4	<code>test_connection()</code>	The in-app “Test connection” helper works
5	STK push	A live Lipa-na-M-Pesa prompt is accepted

Exit code `0` = all passed. A PASS on check 5 means the borrower’s phone receives the STK prompt; the **final** result then arrives asynchronously on the `CallbackURL`, which requires a publicly reachable callback host (`DARAJA_CALLBACK_BASE_URL`) — see §3.

3. Optionally target Safaricom’s documented sandbox test number explicitly:

```
bash
TEST_MSISDN=254708374149 DARAJA_CONSUMER_KEY=xxxx \
DARAJA_CONSUMER_SECRET=yyyy python3 scripts/test_daraja_sandbox.py
```

Once every check passes, the same code path is proven — going to production is then purely the credential/host swap described below.

2. Obtain production credentials (Safaricom Daraja portal)

Do this on <https://developer.safaricom.co.ke> with the organisation's Daraja account (the one that owns the production shortcode).

1. **Create / select a production app** and go through Safaricom's **Go Live** process. This yields a **production** Consumer Key and Consumer Secret (distinct from the sandbox pair).
2. **Production shortcode / B2C paybill.** Confirm the organisation's live B2C shortcode (PartyA for disbursements). For collections you also need the paybill/till used for C2B and STK.
3. **Initiator (API operator).** In the M-Pesa Org Portal, create/identify the **initiator username** with the **Business Payment** (B2C) role and its plaintext initiator password.
4. **Generate the Security Credential.** Encrypt the initiator's plaintext password with **Safaricom's PRODUCTION public certificate** (the sandbox cert will not work in production):
 - Download the production certificate from the Daraja portal.
 - Encrypt with RSA (PKCS#1 v1.5) and Base64-encode the result. Safaricom's portal provides a "Security Credential" generator; or do it locally with the production `.cer` file. The resulting Base64 string is `DARAJA_SECURITY_CREDENTIAL`.
5. **Passkey** (`DARAJA_PASSKEY`) — required only for STK push (collections); obtain the production Lipa-na-M-Pesa passkey for the paybill.

Keep every one of these values out of git — they go only in `backend/.env` on the server (which is git-ignored) or the platform secret store.

3. Whitelist the callback URLs on Daraja

Safaricom must be able to POST asynchronous results back to this app. All callbacks route to the platform host in `DARAJA_CALLBACK_BASE_URL` and carry the unguessable `MPESA_CALLBACK_TOKEN` as a path segment. The **exact endpoints this app exposes** (defined in `backend/app/routers/payments.py`, prefix `/api/v1/payments`) are:

Purpose	Method	Path
B2C Result URL	POST	/api/v1/payments/mpesa/{MPESA_CALLBACK_TOKEN}/b2c-result
B2C Queue Timeout URL	POST	/api/v1/payments/mpesa/{MPESA_CALLBACK_TOKEN}/b2c-timeout
STK push Callback URL	POST	/api/v1/payments/mpesa/{MPESA_CALLBACK_TOKEN}/stk-callback
C2B Validation/Confirmation	POST	/api/v1/payments/mpesa/{MPESA_CALLBACK_TOKEN}/c2b-callback

Full public URLs to whitelist (substitute the real host and token):

```
https://<DARAJA_CALLBACK_BASE_URL>/api/v1/payments/mpesa/<MPESA_CALLBACK_TOKEN>/b2c-result
https://<DARAJA_CALLBACK_BASE_URL>/api/v1/payments/mpesa/<MPESA_CALLBACK_TOKEN>/b2c-timeout
https://<DARAJA_CALLBACK_BASE_URL>/api/v1/payments/mpesa/<MPESA_CALLBACK_TOKEN>/stk-callback
https://<DARAJA_CALLBACK_BASE_URL>/api/v1/payments/mpesa/<MPESA_CALLBACK_TOKEN>/c2b-callback
```

Notes:

- The app builds these automatically (`mpesa.callback_url()`), so once `DARAJA_CALLBACK_BASE_URL` and `MPESA_CALLBACK_TOKEN` are set, the B2C payload and C2B registration already point Safaricom at the right places.
- Register the **B2C Result URL and Queue Timeout URL** against the B2C shortcode in the M-Pesa Org Portal / your app config.
- Register the **C2B Validation & Confirmation URLs** via the C2B `registerurl` call (the app's `register_c2b_urls()` does this) or the portal.
- The STK **CallbackURL** is sent inline on each STK push — no pre-registration, but Safaricom must be able to reach it (public HTTPS).
- Safaricom sends **no auth header** on webhooks; the `MPESA_CALLBACK_TOKEN` path segment is the source-auth control. Pair it with the Safaricom IP allow-list in `deploy/finyln-dcp.conf` (nginx) for defence-in-depth.

4. Set the environment variables

On the server, edit `backend/.env` (never commit it). Set the production values and flip the switch **last**:

```
DARAJA_ENVIRONMENT=production
DARAJA_CONSUMER_KEY=<prod consumer key>
DARAJA_CONSUMER_SECRET=<prod consumer secret>
DARAJA_SHORTCODE=<prod B2C shortcode>
DARAJA_INITIATOR_NAME=<prod initiator username>
DARAJA_SECURITY_CREDENTIAL=<Base64 security credential (prod cert)>
DARAJA_PASSKEY=<prod Lipa-na-M-Pesa passkey> # STK only
DARAJA_CALLBACK_BASE_URL=https://<your production host>
MPESA_CALLBACK_TOKEN=<long random string, unique per environment>
```

Per-DCP (per-tenant) credentials configured in the in-app Configuration screen continue to override these platform defaults field-by-field, and their `environment` value participates in the same base-URL selection.

5. Restart the service

```
sudo systemctl restart finyl-dcp
journalctl -u finyl-dcp -n 50 --no-pager
```

Confirm the boot log shows:

```
Daraja environment: production (base_url=https://api.safaricom.co.ke,
status=LIVE); ...
```

and that all `configured: yes` booleans are `yes`. If the log still says `sandbox`, the env var did not load — fix `backend/.env` and restart before proceeding.

6. Controlled first live disbursement test

Perform ONE small, controlled real payout to a phone you control (e.g. KES 10) before enabling normal operations:

1. In the app, create/approve a tiny loan (smallest allowed principal) to a borrower phone number **you control**.
2. Trigger the disbursement (B2C). Watch the logs:


```
bash
journalctl -u finyl-dcp -f
```
3. Expect the loan to move `approved` → `processing` and an async acknowledgement (`ConversationID`) to be recorded.
4. Confirm Safaricom POSTs the **b2c-result** callback and the loan settles `processing` → `active` (or reverts to `approved` on failure). Check the phone actually received the funds.
5. Verify the transaction appears under Payments → Transactions with a real M-Pesa receipt.

If the payout does **not** settle, check: callback URLs whitelisted & reachable, `MPESA_CALLBACK_TOKEN` matches the registered URLs, nginx IP allow-list not blocking Safaricom, and the initiator has the Business Payment role.

7. Webhook hardening (multi-paybill production pilot)

The callback pipeline is hardened for production with three **additive** layers. All are on by default in a safe mode; only the IP allowlist needs a deliberate flip at go-live.

7.1 Safaricom perimeter IP allowlist

An application-layer allowlist runs **only** on the `/mpesa/{token}/*` callbacks (defence-in-depth on top of the nginx allow-list and the secret path token). It resolves the real client IP behind nginx from `X-Forwarded-For` (first hop) / `X-Real-IP`, and decides using `SAFARICOM_IP_ENFORCE`:

SAFARICOM_IP_ENFORCE	Behaviour
<code>off</code>	Skip the check entirely.
<code>log (default)</code>	Non-allowlisted IP is processed but logged as a warning.
<code>enforce</code>	Non-allowlisted IP is rejected with HTTP 403 before processing.

The allowed ranges come from `SAFARICOM_IP_ALLOWLIST` (comma-separated CIDRs; ships with the known Safaricom production ranges). Only the client IP + endpoint label are ever logged — **never** the payload or any secret.

Go-live action: set `SAFARICOM_IP_ENFORCE=enforce` in `backend/.env` once you have confirmed (from a few days of `log`-mode warnings) that genuine Safaricom callbacks all originate from the configured ranges. Sandbox/testing stays on the default `log` so nothing is ever dropped there. If Safaricom publishes new ranges, add them to `SAFARICOM_IP_ALLOWLIST` **before** flipping to `enforce`.

7.2 Durable ingestion, dead-letter queue & retry

Every callback is now persisted to `mpesa_webhook_events` (migration `016_webhook_durability.sql`) as `received` **before** any processing, so no delivery is ever lost. The existing (unchanged) money logic then runs inside a guard:

- **success** → event marked `processed` ;
- **failure / unresolved** (exception, or no matching txn/loan/tenant) → the DB work is rolled back and the event marked `failed` with an **exponential-backoff** **retry** scheduled (`WEBHOOK_RETRY_BASE_SECONDS` , doubling, capped at 1h);

- after `WEBHOOK_MAX_ATTEMPTS` (default 5) → event escalated to `dead` with a structured **ERROR alert** log line (`ALERT daraja_webhook_dead_letter ...`).

Safaricom always receives HTTP 200 (its expected ack) even on an internal failure, so Safaricom does not retry while our durable queue owns the retry. Retries re-run the **same idempotent processors** the live callbacks use, so a retry can never double-credit a loan.

Two background workers run inside the existing in-process scheduler (`SCHEDULER_ENABLED=true`):

- `webhook_retry` (every 2 min) — reprocesses due `failed` events.
- `webhook_purge` (every 60 min) — NULLs `raw_payload` of processed events older than `WEBHOOK_RAW_RETENTION_HOURS` (default 168h = 7 days), keeping only non-PII metadata for audit (ODPC data-minimisation). Failed/dead events retain their body until resolved.

Monitoring the DLQ: `GET /api/v1/admin/webhook-health` (`super_admin`) returns event counts by status (`received` / `processed` / `failed` / `dead`) plus the most recent dead-lettered events (metadata only, no payload). Alert when `dead` > 0.

7.3 Robust multi-paybill tenant resolution

Callbacks are attributed to the owning tenant by the stored transaction (B2C / STK) or by `BusinessShortCode` → `tenant` (C2B/B2C) via each tenant's `TenantIntegrationConfig` (`integration='daraja'` , `config.shortcode`). A shortcode claimed by two tenants is treated as **ambiguous** → **unresolved** (never guessed). An unroutable callback is recorded with `tenant_id = NULL` + status `failed` and alerted — **never silently dropped or misrouted**.

8. Pre-go-live checklist

- [] Production Daraja app created; **production** consumer key & secret in hand.
- [] Production B2C shortcode confirmed.
- [] Initiator username created with **Business Payment (B2C)** role.
- [] `DARAJA_SECURITY_CREDENTIAL` generated with the **production** certificate (not sandbox).
- [] Production STK passkey obtained (if collections/STK is used).
- [] `DARAJA_CALLBACK_BASE_URL` points at the real public HTTPS host.
- [] `MPESA_CALLBACK_TOKEN` set to a fresh long random value (unique to prod).
- [] All four callback URLs whitelisted/registered on Daraja (see §3).
- [] nginx Safaricom IP allow-list reviewed (`deploy/finyl-dcp.conf`).
- [] `SAFARICOM_IP_ALLOWLIST` reviewed against current Safaricom ranges, then `SAFARICOM_IP_ENFORCE=enforce` flipped (after a few days in `log` mode show all genuine callbacks are in-range). See §7.1.
- [] Migration `016_webhook_durability.sql` applied (creates `mpesa_webhook_events`); `SCHEDULER_ENABLED=true` so retry/purge workers run.
- [] Each pilot tenant's paybill saved in `TenantIntegrationConfig` (`integration='daraja'` , `config.shortcode`) for multi-paybill routing (§7.3).
- [] DLQ monitoring wired: alert on `GET /api/v1/admin/webhook-health` `dead` > 0.

- [] backend/.env updated; .env is **NOT committed to git**.
- [] DARAJA_ENVIRONMENT=production flipped **last**, after all creds are set.
- [] Service restarted; boot log shows environment: production + all configured: yes .
- [] Controlled KES-10 live disbursement completed and settled successfully.
- [] Rollback understood: set DARAJA_ENVIRONMENT=sandbox and restart to instantly and safely revert to sandbox.

9. Rollback

To revert to sandbox at any time (no code change):

```
# in backend/.env
DARAJA_ENVIRONMENT=sandbox
```

```
sudo systemctl restart finyl-dcp
```

Confirm the boot log reports environment: sandbox . Because production fails closed, an incomplete production config never results in a live payout — the worst case is a refused disbursement (HTTP 422), never a misdirected one.